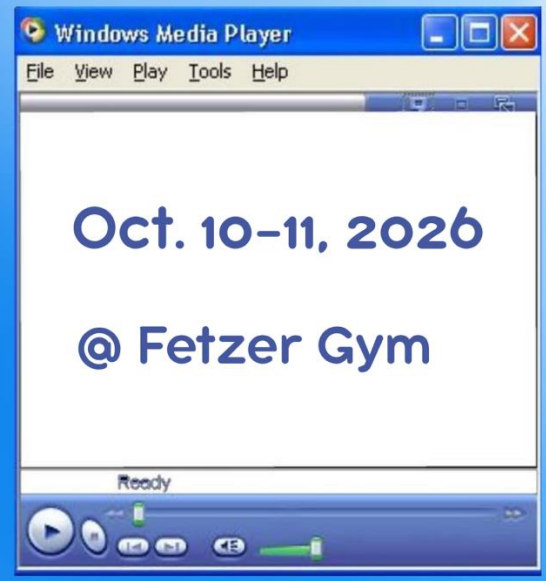


Floating Point

COMP211 Connor McMahon

HACKNC





prizes!

cle credit!

register now !

hacknc.com

@thehacknc

Normalized Bias for Minifloat = 3

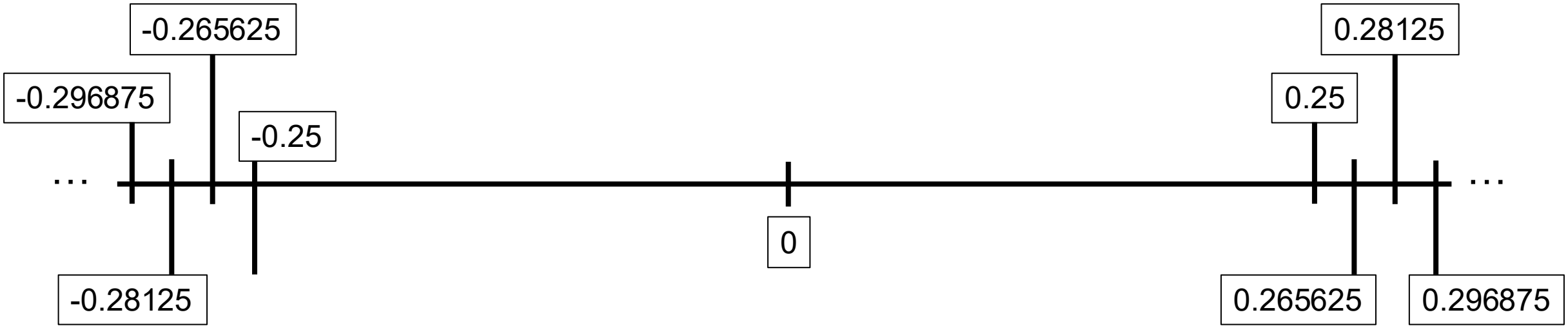
$$\text{Bias} = 2^{n-1}-1$$

- n = number of bits in the exponent field

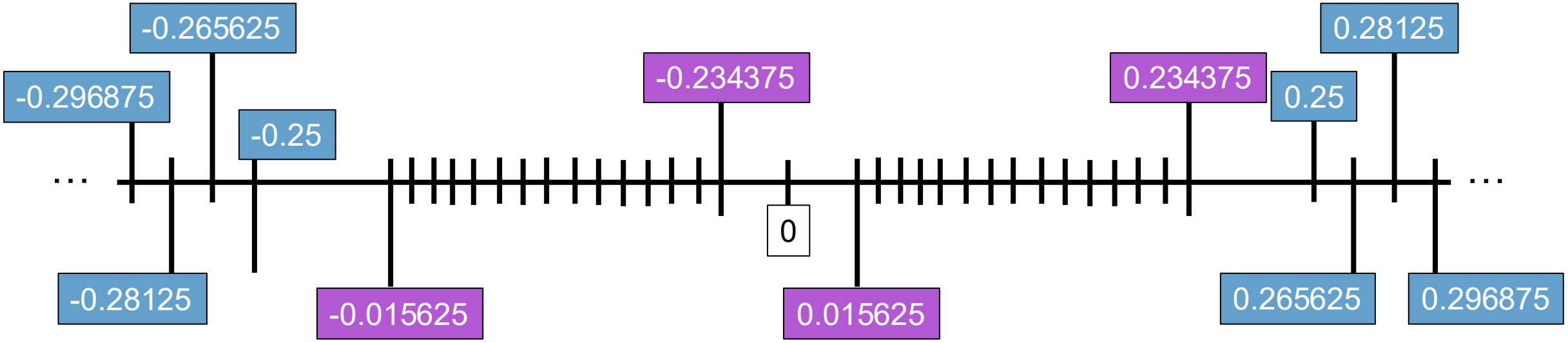
- This allows for the same number of negative and non-negative exponents

Denormalized Numbers

Number Line of Normalized Values



Number Line of Representable Values



= normalized numbers

= denormalized numbers

- **Overflow:** The result of a computation exceeds the largest representable floating-point number
- **Underflow:** A number is too small to be represented and is rounded to zero.

Normalized Exponent

For normalized numbers, the exponent is calculated as:

$$\mathrm{exponent} = \mathrm{exponent\ field} - \mathit{bias}$$

The smallest exponent field available to a **normalized** number is `001` because `000` is reserved for zero and subnormal numbers. So,

$$\mathrm{exponent} = 001_2 = 1$$

and:

$$\mathrm{exponent} = 1 - 3 = \boxed{-2}$$

Therefore, the smallest normalized numbers have the form:

$1.xxxx_2 \times 2^{-2}$

Denormalized Exponent

When the exponent field is all zeros, the effective exponent is defined as:

$$\boxed{1 - bias}$$

So with a bias of 3:

$$1 - 3 = \boxed{-2}$$

A subnormal therefore has the form:

$$0.xxxx_2 \times 2^{-2}$$

Denormalized Exponent

Notice the important difference:

Smallest Normalized: $1.xxx_2 \times 2^{-2}$
All Subnormal: $0.xxx_2 \times 2^{-2}$

Both use 2^{-2} .What changes is the implicit leading bit: normalized numbers have an implicit **1**, while subnormals have an implicit **0**.

Denormalized Numbers

This makes the transition between subnormal and normalized numbers smooth. With 4 fraction bits, the **largest subnormal** is:

$$0.1111_2 \times 2^{-2}$$

which is:

$$\frac{15}{16} \times \frac{1}{4} = \frac{15}{64}$$

The **smallest normalized** number is:

$$1.0000_2 \times 2^{-2} = \frac{1}{4} = \frac{16}{64}$$

Denormalized Exponent

Here values around the transition from denormalized to normalized values:

```
13/64
14/64
15/64     (largest denormal)
16/64     (smallest normalized)
17/64
18/64
...
```

So they are adjacent:

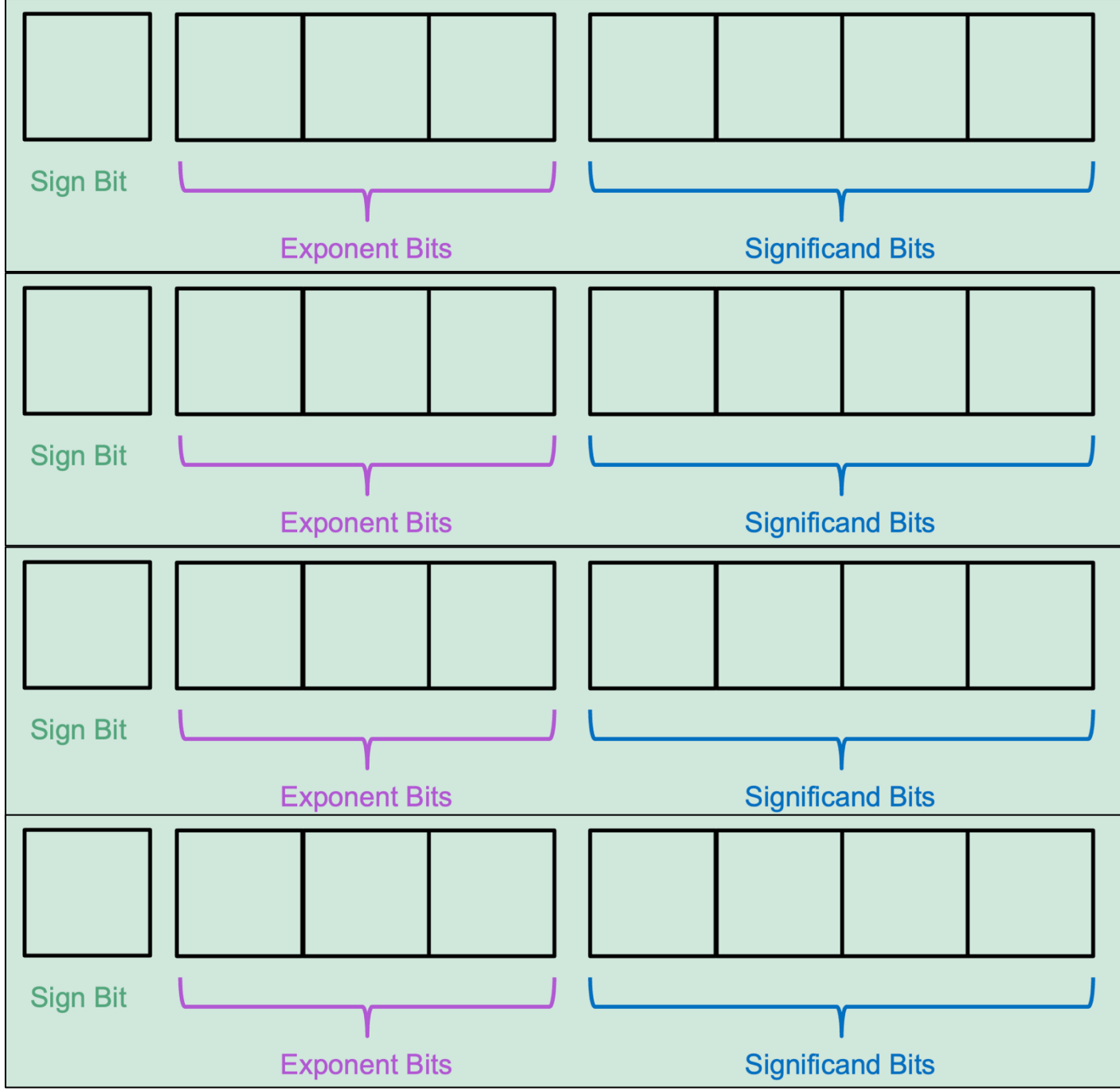
$$\frac{15}{64}, \frac{16}{64}$$

The spacing is exactly:

$$\frac{1}{64}$$

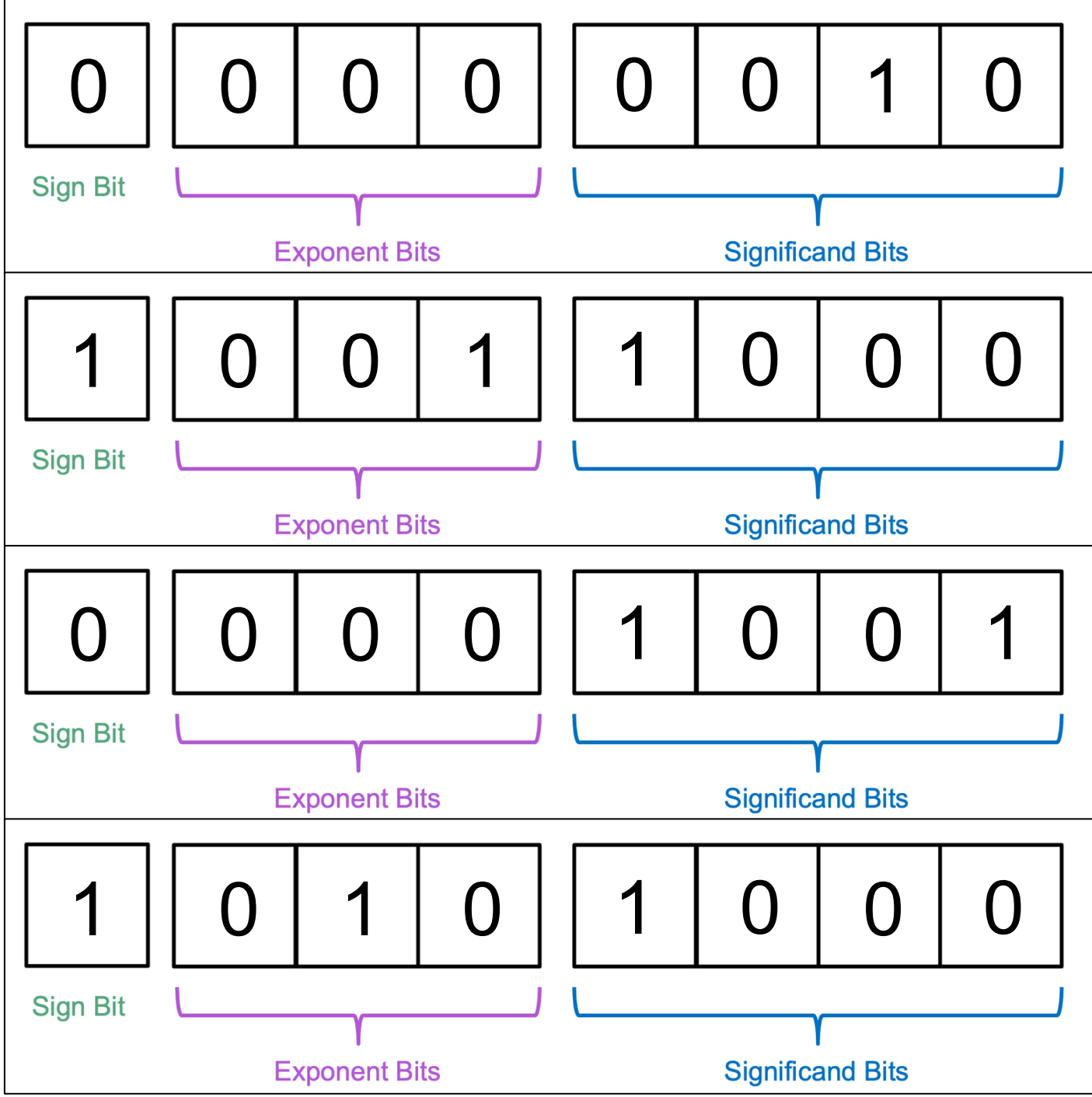
Minifloat

- For each value below, indicate whether it will be represented as a normalized or denormalized value. Then write out its encoding. Hint: first try encoding as a normalized number. If the exponent + bias is <= 0, it is too small to be a normalized number, so try dernorm.
- A) 1/32
- B) -3/8
- C) 9/64
- D) -48/64

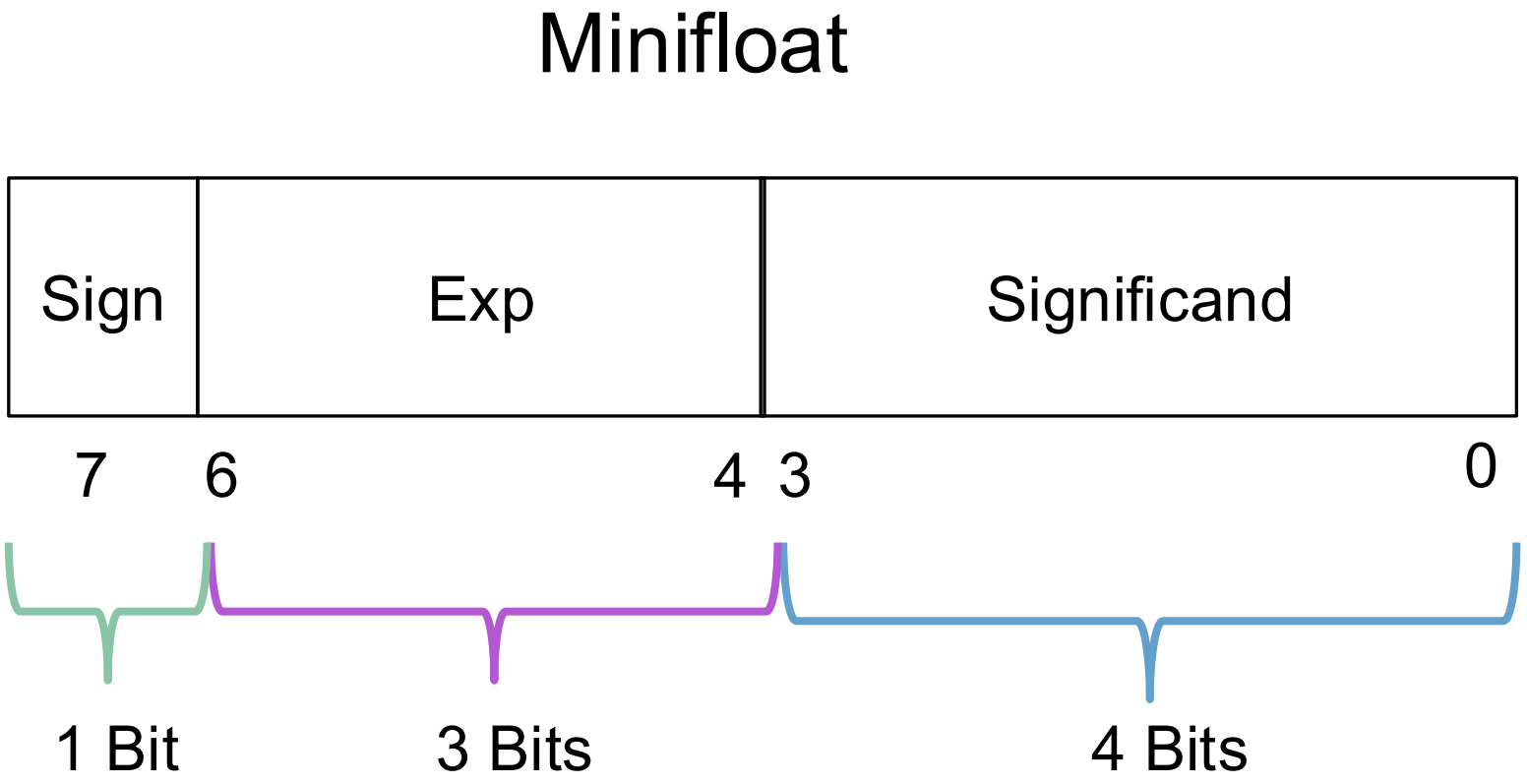


Minifloat

- For each value below, indicate whether it will be represented as a normalized or denormalized value. Then write out its encoding.
- A) 1/32 = 0.00001_2
  - Denorm
  - $0.001_2 \times 2^{-2}$
- B) -3/8 = -(1/4 + 1/8)
  - Normalized
  - $= -0.011_2 = -1.1 \times 2^{-2}$
- C) 9/64 = 8/64 + 1/64
  - Denorm
  - $0.001001 = 0.1001 \times 2^{-2}$
- D) -48/64 = -3/4
  - Normalized
  - $-0.11 = -1.1 \times 2^{-1}$



Notation



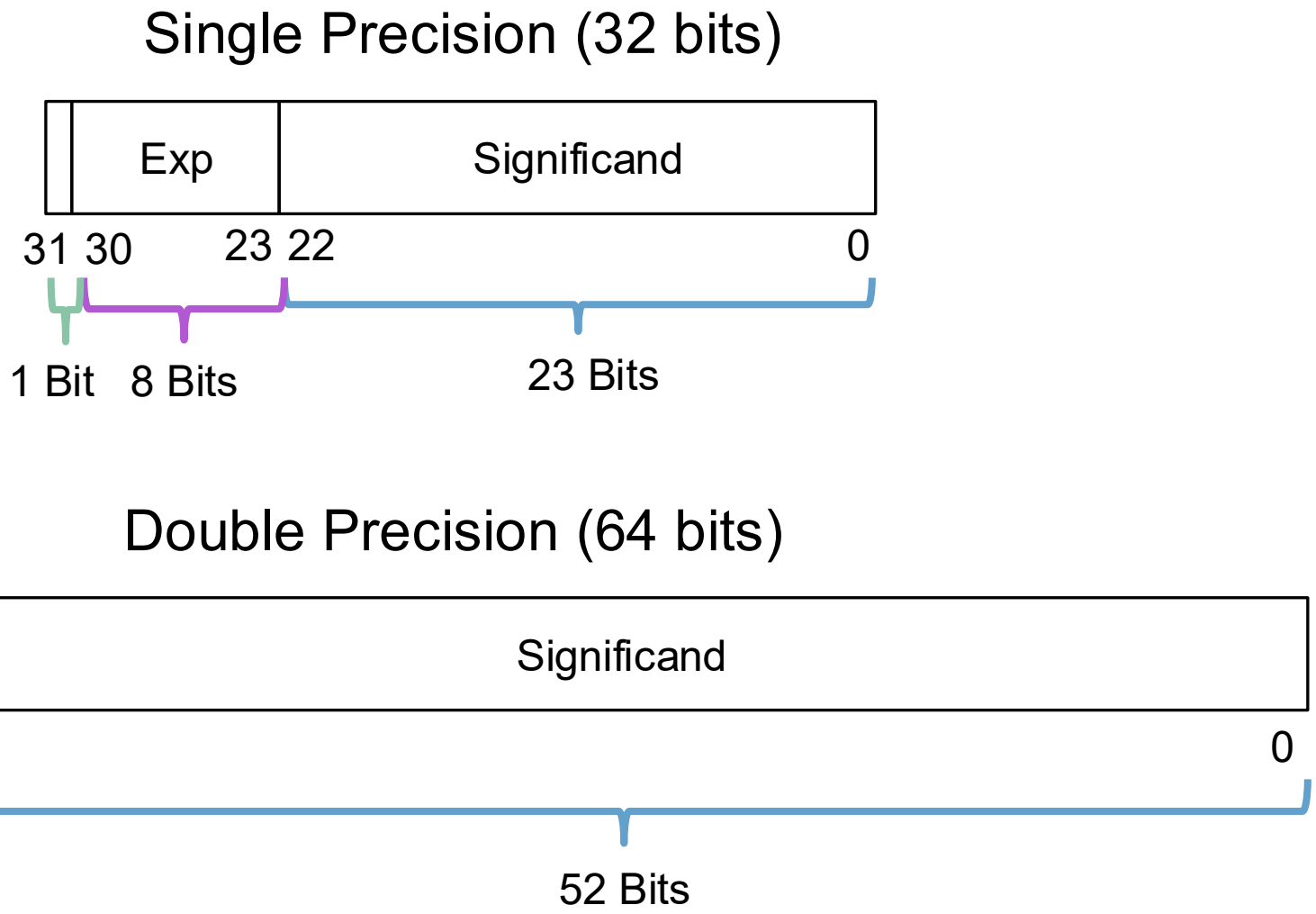
IEEE-754

IEEE-754 Floating Point

- Floating point standard established in 1985
  - Effectively all modern floating-point implementations use the standard!
- **Single-precision** Floating Point: **32-bits**
  - This is a **`float`** in C, Java, and so on
- **Double-precision** Floating Point: **64-bits**
  - This is a **`double`** in C, Java, and so on

	Single Precision	Double Precision
Sign Bit	1	1
Exponent Bits	8	11
Bias	127	1023
Significand Bits	23	52

IEEE-754 Floating Point



Single vs Double Precision

Aspect	Single Precision (32 bits)	Double Precision (64 bits)
Accuracy	Smaller range and less precision => less accurate	Larger range and more precision => more accurate
Storage	Less space	More space
Speed	Faster	Slower

IEEE-754 Floating Point

- The same rules apply as with minifloat

Value	Exponent Bits	Significand Bits
Normalized #	Nonzero	Anything
Zero	All zeros	All zeros
Infinity	All ones	All zeros
NaN	All ones	Nonzero
Denormalized #	All zeros	Nonzero

IEEE-754 Floating Point

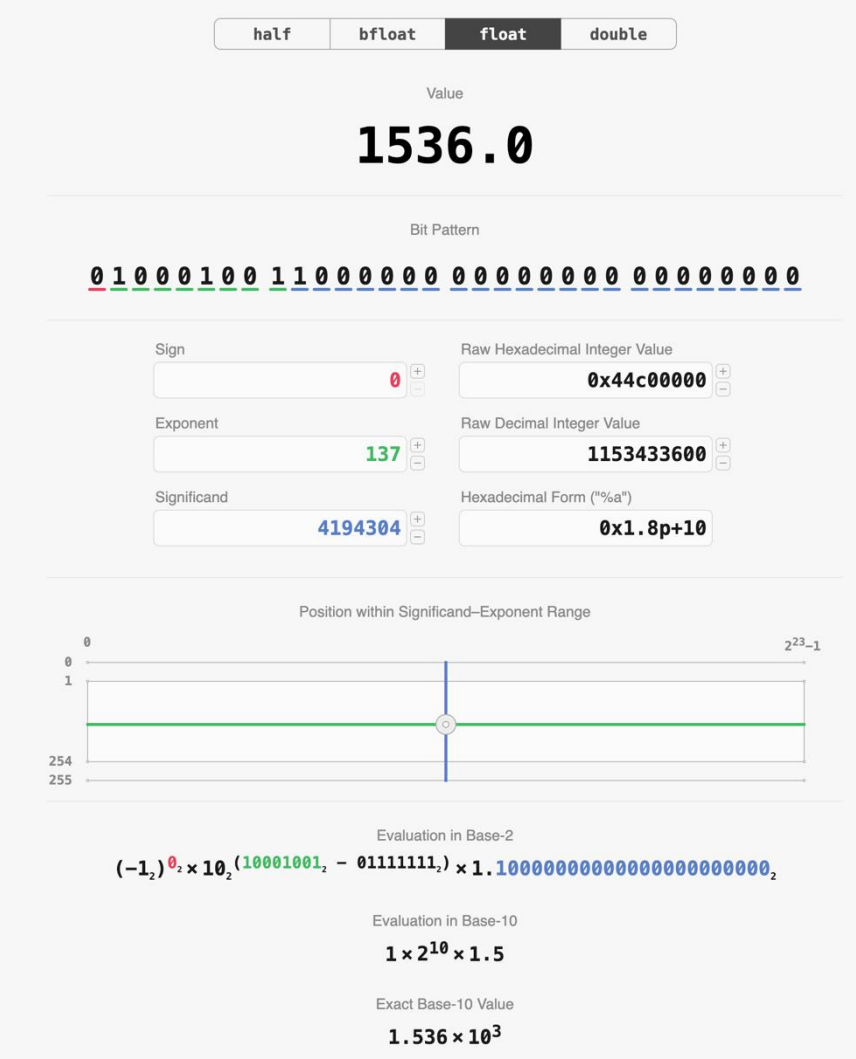
- Inside your processor, there are specialized circuits called floating-point units (FPUs).

- The FPU handles arithmetic (add, subtract, multiply, divide, square root, etc.) directly in hardware, using IEEE 754 rules.

- That means the same floating-point calculation will behave the same way on different machines (with the same precision), which is crucial for consistency in programming, science, and engineering.

Tools for Tinkering and Checking Understanding

- https://float.exposed/0x44c00000



Precision Loss (Truncation Errors)

Rounding Errors

```c
#include <stdio.h>

int main() {
   double a = 0.1;
   double b = 0.2;
   printf("%.23f\n", a);
   printf("%.23f\n", b);
   printf("%.23f\n", a + b);
}
```

We can see that 0.1 + 0.2 != 0.3

Do not check for equality using ==

```
0.10000000000000000555112
0.20000000000000001110223
0.30000000000000004440892
```

Since floating-point numbers have a limited number of bits, many real numbers cannot be represented exactly and must be rounded to the nearest representable value.

Floating Point Equality

- Floating-point numbers **should not** be compared using the == operator due to rounding errors and precision loss!
- Instead, comparisons should use an **epsilon-based approach** to check if two values are *close enough*.
- An **epsilon** (ϵ) is a small threshold value used to determine if two floating-point numbers are "close enough" to be considered equal.
- The C standard library (float.h) provides machine-specific precision constants:
  - FLT_EPSILON (1.19209290e-7F) for float
  - DBL_EPSILON (2.2204460492503131e-16) for double

Floating Point Equality

```
#include <float.h>
#include <math.h>
#include <stdio.h>

int are_equal(double a, double b) {
        return fabs(a - b) < DBL_EPSILON;
}

int main() {
        double x = 0.1;
        double y = 0.2;
        double z = 0.1 + 0.2;
        double exp_res = 0.3;

        printf(”x: %.23f: \n", x);
        printf(”y: %.23f: \n", y);
        printf(”z: %.23f: \n", z);
        printf(”exp_res: %.23f: \n\n", exp_res);

        if (are_equal(z, exp_res)) {
                printf("x and y are approximately equal\n");
        } else
                printf("x and y are NOT equal\n");
        }
        return 0;
}
```

```
x: 0.10000000000000000555112
y: 0.20000000000000001110223
z: 0.30000000000000004440892
exp_res: 0.29999999999999998889777

x and y are approximately equal
```

Exercise

```c
#include <float.h>
#include <stdio.h>

int main(void) {
    float x = 1.0;
    float delta = 0.1;

    printf("\nx = %.23f\n", x);
    printf("delta = %.23f\n", delta);
    printf("x + delta = %.23f\n", x + delta);

    if (x + delta == x) {
        printf("\nAdding delta had NO effect!\n\n");
    } else {
        printf("\nAdding delta changed x.\n\n");
    }

    return 0;
}
```

Exercise

- **Q1:** Find a value of delta that is **not zero**, but is small enough that adding it to x has no effect.
  - Start with: float delta = 0.1;
  - Run the program. Then make delta smaller and run it again.
  - For example:
    - 0.1
    - 0.01
    - 0.001
    - ...
  - Q1.1 What is the smallest power of 10 that has no effect?
  - Q1.2 delta is not zero. Why does adding it to x have no effect?
- **Q2:** Then, change `x` and `delta` to `doubles` and repeat the same experiment.
- **Q3:** Why can a double add a much smaller delta to 1.0 before the addition stops having an effect?

Exercise



- Q1.1: For float:
  - delta = 0.00000001; // 1e-8
- Q1.2: Both x and delta are representable. The problem is that there is not enough precision to represent x + delta.

- Q2: For double:
  - delta = 0.0000000000000001; // 1e-16

- Q3: A double has **52 fraction bits**, while a float has **23 fraction bits**. Therefore, representable double values around 1.0 are much more closely spaced. A much smaller change can therefore produce a different representable value.

Real-World Application
Floating Point Numbers in LLMs

Floating Point Big Ideas

- There is a tradeoff between range and precision
- Not every real number is representable
- Arithmetic may round, overflow, or underflow

What is an LLM?

- A **Large Language Model (LLM)** is a program that has been trained to recognize patterns in large amounts of text.
- Given some text, it predicts what should come next.
- For example, given:
  - The capital of France is ___
  - the model might assign probabilities like:
    - Paris 97%
    - London 1%
    - France 0.5%
    - ...
- It then uses these predictions to generate text one piece at a time.

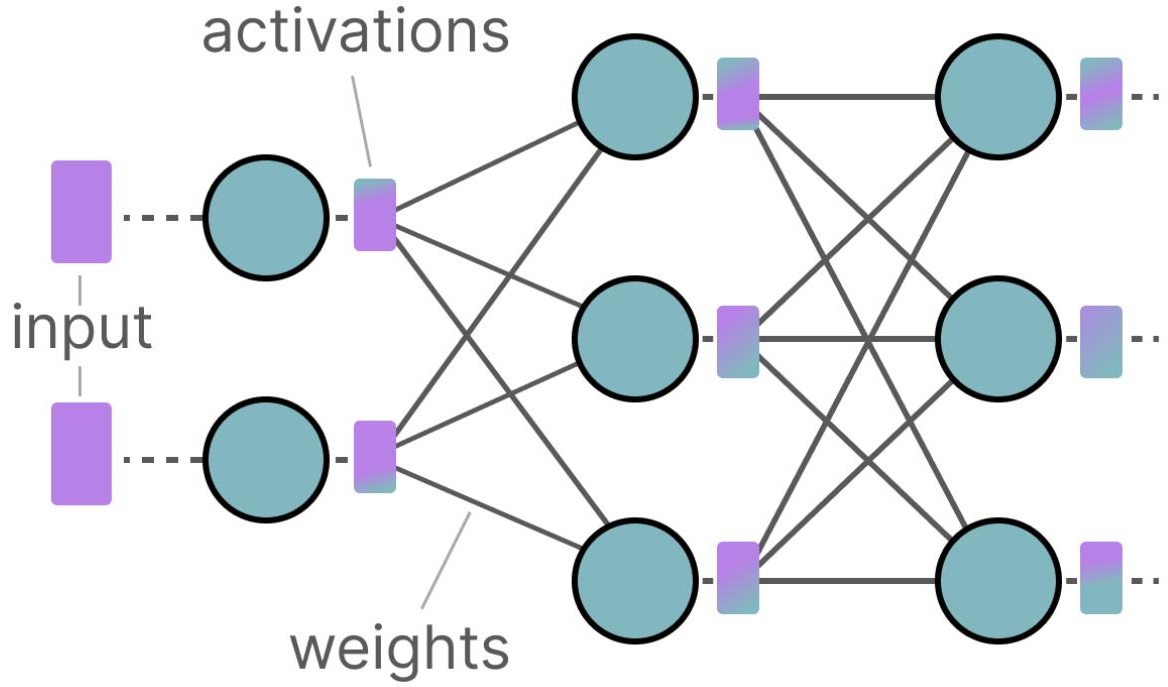
How does it learn these predictions?

- An LLM contains billions of numerical values called parameters (or **weights**).
  - You can think of these values as numbers that control how strongly different patterns influence the model's predictions.

- During **training**, the model sees many examples of text, makes predictions, measures how wrong those predictions are, and adjusts its parameters.

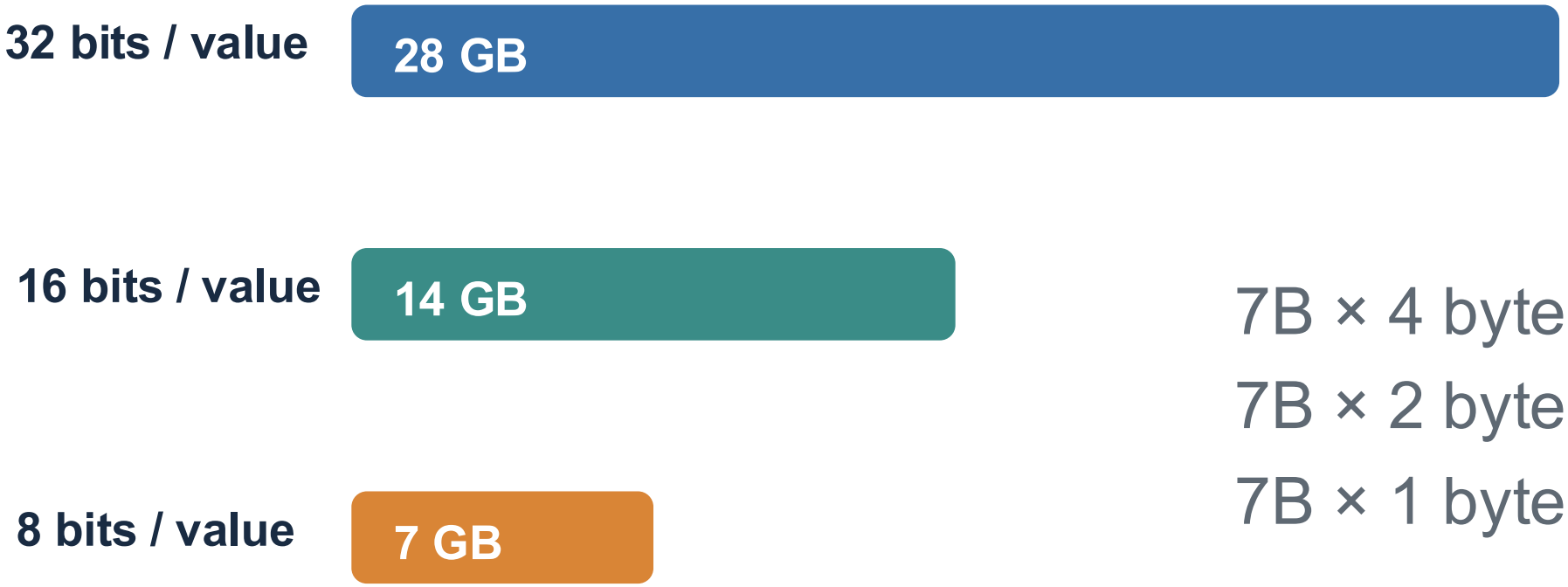
Where are the floating point numbers?

- Weights
  - Numbers learned during training
  - A large model contains billions of these values
- Activations
  - Temporary values stored while processing your input
  - They change for every prompt and every layer



Why do the number of bits matter?

- Imagine a model with 7 billion parameters



7B × 4 bytes ≈ 28 GB

7B × 2 bytes ≈ 14 GB

7B × 1 byte ≈ 7 GB

Common Data Types: FP16

- **FP16** stands for **16-bit floating point** (also called **half precision**).
- Like the floating-point representations we've been using, FP16 divides its bits into three fields:

Field	Bits	Purpose
Sign	**1**	Positive or negative
Exponent	**5**	Controls the range
Fraction	**10**	Controls the precision

- For a normalized FP16 number: $(-1)^S \times 1.F \times 2^{E-bias}$
  - The bias for FP16 is 15
  - So, for example, if the exponent field contains:
    - $10000_2 = 16$
  - the actual exponent is:
    - 16−15=1

Common Data Types: FP16

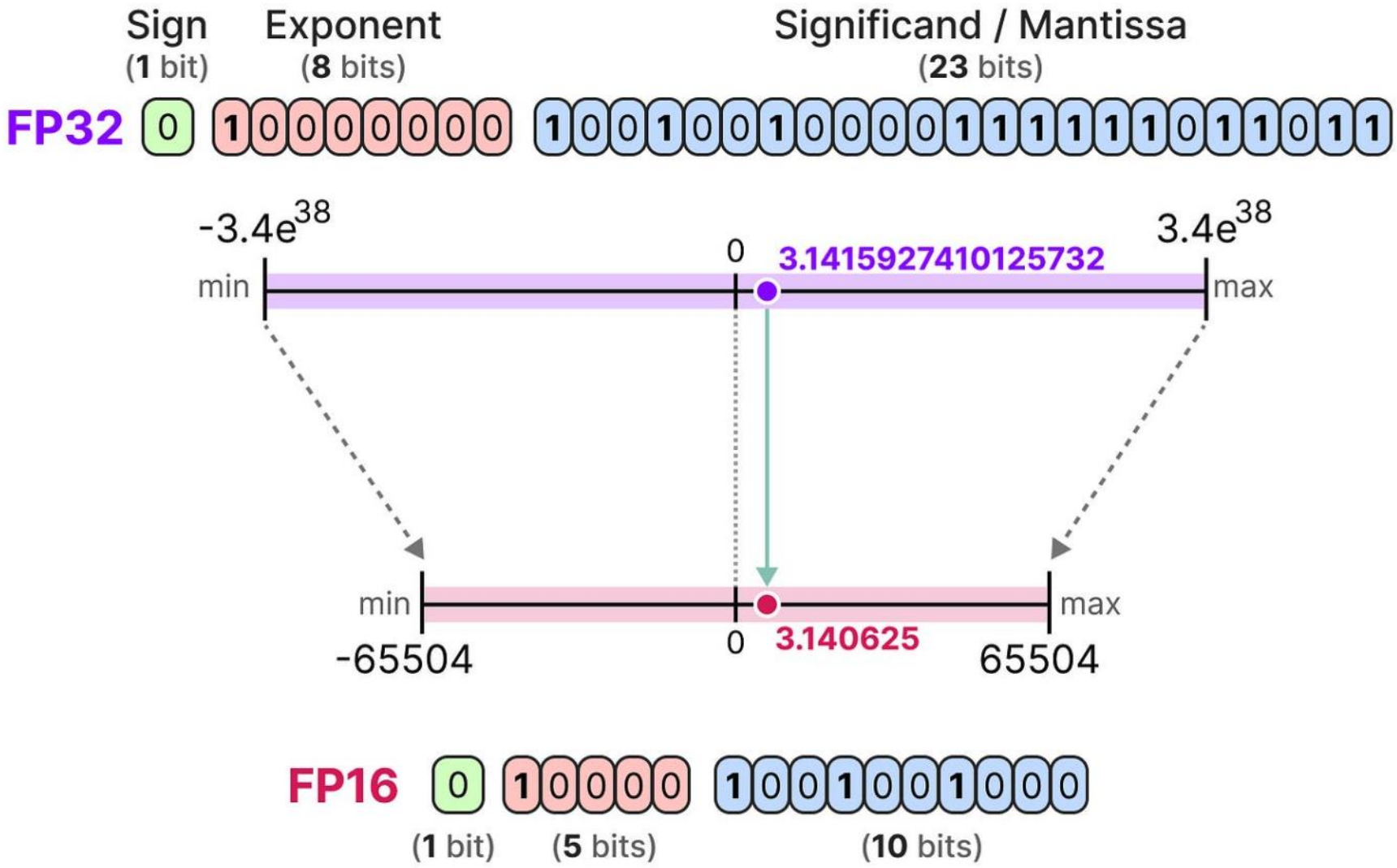


Image source: https://www.maartengrootendorst.com/blog/quantization/

Why use FP16 instead of FP32?

- Imagine an LLM has 7 billion parameters.
  - If they're all FP32:
    - $7\ billion \times 4\ bytes \approx 28\ GB$
  - If they're all FP16:
    - $7\ billion \times 2\ bytes \approx 14\ GB$
  - That's a huge difference when you're moving billions of values between memory and processors.
- **But there's a tradeoff!!!**
  - FP16 saves space by having fewer bits available for the exponent and fraction.
  - So FP16 gives us **less range and less precision** than FP32.
  - That means more numbers have to be rounded, and very large values can overflow more easily.

Common Data Types: FP16

- For many LLM computations, the benefit of storing and operating on smaller values can outweigh the loss of precision.

- FP16 can mean
  - less memory usage
  - less memory traffic
  - hardware that can perform more low-precision operations efficiently.

Common Data Types: BF16

- **BF16** stands for **Brain Floating Point 16** (bfloat16).
- Designed for neural networks
- Like FP16, it uses **16 total bits**, but it divides those bits differently:

Field	FP16	BF16
Sign	1	**1**
Exponent	5	**8**
Fraction	10	**7**
Total	16	**16**

- **FP16** chooses: more *precision, less range*
- **BF16** chooses: more *range, less precision*

BF16

- **Why 8 exponent bits?**
- There's an especially useful connection to FP32.
  - FP32 uses:
    - 1 sign | 8 exponent | 23 fraction
  - BF16 uses:
    - 1 sign | 8 exponent | 7 fraction
- They have the **same number of exponent bits**.
- They also both use an exponent bias of: $\boxed{127}$
- So BF16 has approximately the **same numerical range as FP32**, but much less precision.

Common Data Types: BF16

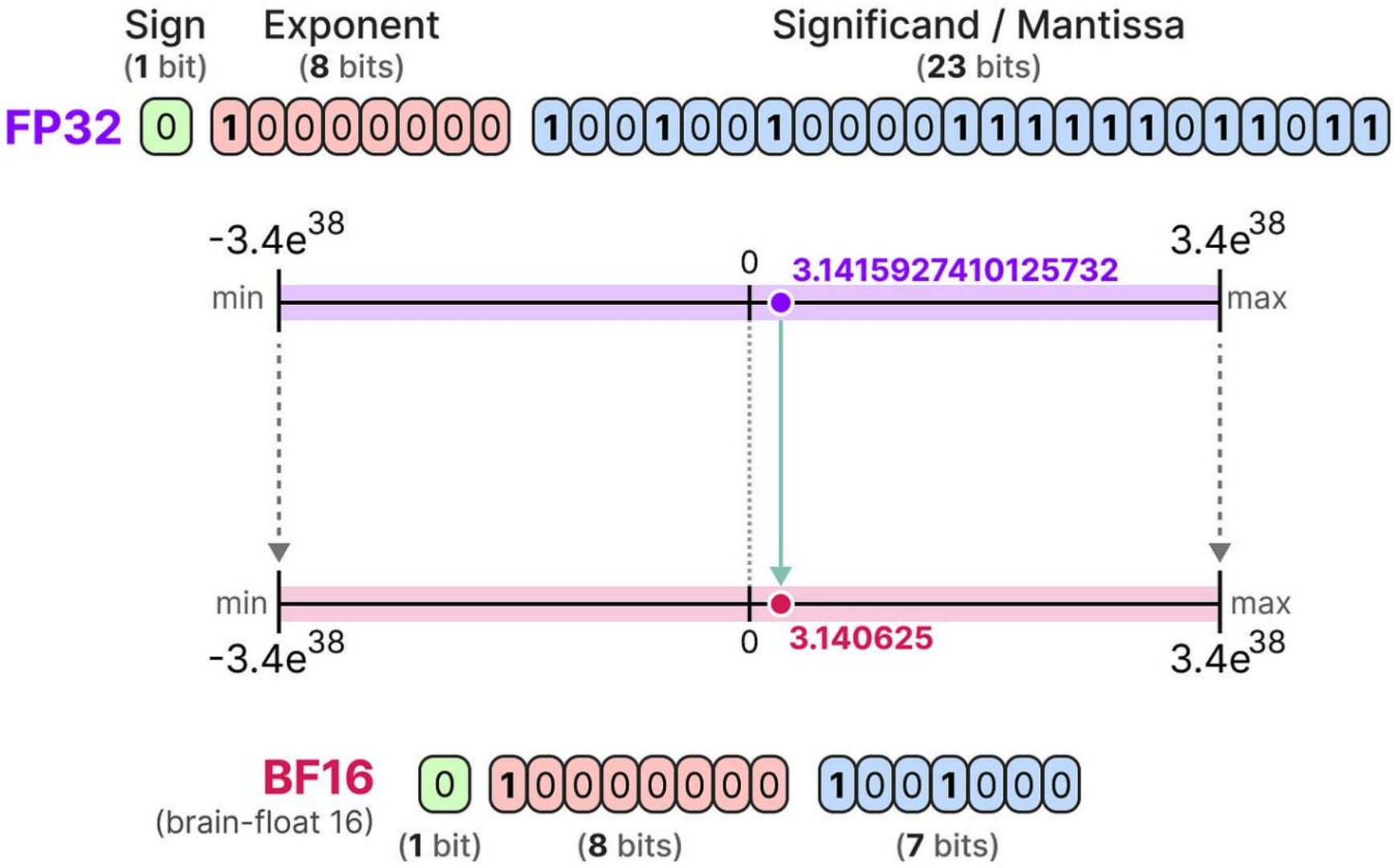


Image source: https://www.maartengrootendorst.com/blog/quantization/

Why would an LLM want more range?

- During neural-network calculations, values can vary considerably in magnitude.
- If the exponent isn't large enough, you can encounter:
  - **Overflow**: a value is too large to represent.
  - **Underflow**: a value is too small to represent normally.
- BF16's 8-bit exponent gives it much more range than FP16.
- For example, the largest finite positive values are roughly:
  - FP16 *max* $\approx 65{,}504$
  - BF16 *max* $\approx 3.39 \times 10^{38}$
  - That's an enormous difference in range!!!

Scenario

- Your team wants to run an LLM with **8 billion parameters** on a computer with limited memory. Each parameter is a number that must be stored in memory. Your hardware can only store 16-bit values. **Fill in the table to help your team understand the tradeoffs.**

Consideration	FP16	BF16
Memory required		
Greater range		
Greater precision		
More likely to overflow		
More rounding		

Scenario



- Your team wants to run an LLM with **8 billion parameters** on a computer with limited memory. Each parameter is a number that must be stored in memory. Your hardware can only store 16-bit values. **Fill in the table to help your team understand the tradeoffs.**

Consideration	FP16	BF16
Memory required	16 GB	16 GB
Greater range		X
Greater precision	X	
More likely to overflow	X	
More rounding		X